## Using prepend

Ruby allows you to place a module at a different point in the method lookup chain using the prepend method. Logically, prepend behaves just like include, but the methods in the prepended module take precedence over those in the host class. Ruby pulls off this magic by inserting a dummy class in place of the original host class. Actually, it inserts the dummy class above the original class, and then moves the methods from the original to the copy. and then inserting the prepended module between the two.

If a method inside a prepended module has the same name as one in the original class, it will be invoked instead of the original. The prepended method can then call the original using super:

```
module VanityPuts
  def puts(*args)
    args.each do |arg|
      super("Dave says: #{arg}")
    end
  end
end

class Object
  prepend VanityPuts
end

puts "Hello and", "goodbye"
```

*produces:*

```
Dave says: Hello and
Dave says: goodbye
```

However, there is a problem with this—the change we just made to class Object is global. We'll see how to manage that shortly when we look at refinements.

## Using extend

The include method effectively adds a module as a superclass of self. It is used inside a class definition to make the instance methods in the module available to instances of the class.

However, it is sometimes useful to add the instance methods directly to a particular object and not to its class. For example, you'd want to do this to add methods directly to a class object that you want to be treated like class methods. You do this using extend. Here's an example:

```
module Humor
  def tickle
    "#{self} says hee, hee!"
  end
end

obj = "Grouchy"
obj.extend(Humor)
obj.tickle # => "Grouchy says hee, hee!"
```

Stop for a second to think about how this might be implemented.